

Phase 1: Base Operating System Setup

1. Install **Ubuntu Desktop LTS** on the Chromebox.
2. During the installation steps, ensure you check the box for **"Log in automatically."**
3. Once booted to the desktop, open **Settings** → **Power** and disable **Screen Blanking / Automatic Suspend** so the screen never goes to sleep.

Phase 2: System Packages & Firewall Setup

Open a terminal (**Ctrl + Alt + T**) and paste this block to update the system, open the network ports for AirPlay mirroring, and install the Docker engine:

Bash

```
# 1. Update the operating system packages
sudo apt update && sudo apt full-upgrade -y
```

```
# 2. Open standard firewall ports for UxPlay mirroring & mDNS discovery
sudo ufw allow 5353/udp
sudo ufw allow 7000:7002/tcp
sudo ufw allow 7000:7002/udp
sudo ufw reload
```

```
# 3. Install AirPlay streaming dependencies and the Avahi network daemon
sudo apt-get install -y ca-certificates curl avahi-daemon gstreamer1.0-plugins-bad
gstreamer1.0-libav gstreamer1.0-vaapi
```

```
# 4. Install the official Docker Engine and Compose plugins
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

```
# 5. Enable Docker to start on system boot
sudo systemctl enable docker.service
sudo systemctl enable containerd.service
```

```
# 6. Authorize your user account to manage Docker containers without sudo
```

```
sudo usermod -aG docker $USER
```

 **CRITICAL:** You must **reboot the Chromebox** (`sudo reboot`) right now before proceeding to Phase 3, or the user permissions won't apply and the following steps will throw permission errors.

Phase 3: Deploy the Signage Stack (Anthias)

Once rebooted, open your terminal back up and build the local digital signage container engine using our customized, hardcoded settings.

1. Create the screenly folder structure:
2. Bash

```
mkdir -p ~/screenly && cd ~/screenly
```

- 3.
- 4.
5. Open a new configuration file:
6. Bash

```
nano docker-compose.yml
```

- 7.
- 8.
9. Paste this exact configuration block, which uses the correct **x86 images**, fixes the memory issues, and strips out the broken Raspberry Pi driver dependencies:

YAML

```
version: '3'
```

services:

anthias-redis:

```
image: ghcr.io/screenly/anthias-redis:latest-x86
```

```
restart: always
```

```
mem_limit: 512m
```

anthias-server:

```
image: ghcr.io/screenly/anthias-server:latest-x86
```

```
restart: always
```

```
depends_on:
```

```
- anthias-redis
```

```
volumes:
```

```
- ./data:/data
```

```
ports:
  - "80:80"
mem_limit: 512m
```

```
anthias-viewer:
  image: ghcr.io/screenly/anthias-viewer:latest-x86
  restart: always
  shm_size: 128m
  depends_on:
    - anthias-server
  environment:
    - DISPLAY=:0
  volumes:
    - /tmp/.X11-unix:/tmp/.X11-unix
  mem_limit: 512m
```

```
anthias-websocket:
  image: ghcr.io/screenly/anthias-websocket:latest-x86
  restart: always
  depends_on:
    - anthias-server
  mem_limit: 512m
```

4. Press **Ctrl + O**, then **Enter** to save, and **Ctrl + X** to exit the text editor.
5. Launch the containers in background mode:
6. Bash

```
docker compose up -d
```

- 7.
- 8.

Phase 4: Configure the Firefox Proxy Whitelist

Because the network filters block standard web looping, we must whitelist local playback.

1. Open **Firefox** manually.
2. Click the **Menu (three lines)** in the top right corner → **Settings**.
3. Scroll down to the very bottom of the *General* tab and click **Settings...** under **Network Settings**.
4. Find the text box labeled **"No proxy for"** and add this exact string to it:
5. Plaintext

localhost, 127.0.0.1, 192.168.0.0/16, 10.0.0.0/8

- 6.
- 7.
8. Click **OK** and close Firefox.

Phase 5: Automate Screen Mirroring (UxPlay Service)

Create a system service so the AirPlay receiver starts automatically at boot on pinned, unblocked ports.

1. Open a terminal file creator:
2. Bash

```
sudo nano /etc/systemd/system/uxplay.service
```

- 3.
- 4.
5. Paste this configuration block (replace **yourusername** with the actual name of your Chromebox Ubuntu user profile):
6. Plaintext

[Unit]

Description=UxPlay AirPlay Receiver

After=network.target avahi-daemon.service docker.service

[Service]

Type=simple

User=yourusername

Environment=DISPLAY=:0

ExecStart=/usr/bin/uxplay -p 7000 -fs -vsync no

Restart=on-failure

[Install]

WantedBy=graphical.target

- 7.
- 8.
9. Save and close (**Ctrl + O**, **Enter**, **Ctrl + X**).
10. Register and enable the background hook:
11. Bash

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable uxplay.service
```

```
sudo systemctl start uxplay.service
```

- 12.
- 13.

Phase 6: Automate the Fullscreen Kiosk on Boot

Finally, create the startup rule that launches Firefox directly into fullscreen mode after waiting for Docker to finish loading.

1. Make sure the hidden user autostart folder exists:
2. Bash

```
mkdir -p ~/.config/autostart
```

- 3.
- 4.
5. Create the startup config file:
6. Bash

```
nano ~/.config/autostart/anthias-kiosk.desktop
```

- 7.
- 8.
9. Paste this desktop metadata inside:
10. Plaintext

```
[Desktop Entry]
Type=Application
Exec=sh -c "sleep 10 && firefox --kiosk http://127.0.0.1"
Hidden=false
NoDisplay=false
X-GNOME-Autostart-enabled=true
Name=Anthias Signage Kiosk
Comment=Launches local digital signage player after a 10s delay
```

- 11.
- 12.
13. Save and close (**Ctrl + O**, **Enter**, **Ctrl + X**).
14. Give the startup shortcut executable rights:
15. Bash

```
chmod +x ~/.config/autostart/anthias-kiosk.desktop
```

- 16.
- 17.

Final Verification

Give the system a full reboot: `sudo reboot`.

The Chromebox will log in autonomously, sit on the desktop for 10 seconds while Docker initializes the layout, and then snap open into a beautiful full-screen digital signage loop. You can now connect to it via AirPlay at any time or manage your slides by pulling up the device's IP address on your laptop!